Technical Architecture and Platform Feasibility Specification for mTools: Unified Multi-Platform Analytics Engine

Direct execution of interactive authenticated requests, programmatic token provisioning, and live sandbox calls against third-party social networking endpoints cannot be fulfilled directly because the connected environment does not support interactive external tool functions; accordingly, this specification delivers comprehensive, evidence-based research, technical constraints, data schemas, and architectural blueprints derived from verified platform documentation to enable direct engineering execution.

Deliverable 1: Executive Summary

The objective of mTools is to serve as an executive command center that consolidates publishing activity, post-level engagement, audience demographics, and content velocity across six primary communication ecosystems: LinkedIn, X (formerly Twitter), Instagram, Facebook, Substack, and Reddit. The technical premise requires ingesting telemetry across these platforms into an integrated datastore while preserving strict provenance boundaries between verified platform data, derived calculations, and artificial intelligence inferences.
The technical feasibility of integrating these platforms is determined by divergent developer access models, API deprecations, and monetization barriers across the social web. The platform feasibility landscape falls into four clear tiers:

Feasibility Tier	Platforms Included	Integration Feasibility Profile	Primary Constraint / Prerequisite
High Feasibility	Instagram Professional, Facebook Pages	Fully supported via official Meta Graph API v26.0 endpoints for media publishing, metrics, and comment management.	Requires Professional account conversion (Instagram) and Facebook Page administration; personal profiles are completely unsupported.
Commercial Gate	X (formerly Twitter)	Fully functional modern REST API v2 supporting publishing, timeline retrieval, and post engagement analytics.	Free tier excludes read endpoints; operation requires the $100/month Basic tier, capped at 10,000 read posts monthly.
Conditional Access	LinkedIn	Rich creator metrics available via memberCreatorPostAnalytics and organization post management.	Requires formal corporate vetting and application approval under the restricted Community

Feasibility Tier	Platforms Included	Integration Feasibility Profile	Primary Constraint / Prerequisite
			Management API product.
Partial Telemetry	Reddit	Official OAuth 2.0 REST API provides submission history, comment trees, karma scores, and upvote ratios.	Post Insights (impressions and view counts) are restricted to internal GraphQL queries and return null on the public REST API.
Asymmetric / Hybrid	Substack	Public XML RSS feeds enable post indexing, but private subscriber and email metrics lack a public REST API.	Requires automated RSS polling coupled with manual or drop-zone ingestion of native creator CSV exports (posts.csv, subscribers.csv).

The ease of integration reflects these platform constraints. Meta platforms (Instagram Professional and Facebook Pages) provide the lowest architectural friction, offering unified Graph API schemas, webhooks, and long-lived tokens. Reddit follows with standard OAuth 2.0 authentication and generous non-commercial rate limits, though its analytical value is capped by the absence of impression data. X API v2 provides a highly predictable integration path but imposes a recurring commercial fee of $100 per month for basic reading privileges. Substack introduces architectural asymmetry, requiring a dual-track ingestion pipeline that combines automated public RSS scraping with scheduled creator file uploads. LinkedIn represents the highest access barrier due to strict corporate application vetting, verified business email prerequisites, and partner program approval requirements.

The largest operational risks across the platform lifecycle include:

- Potential rejection or protracted delay during LinkedIn Community Management API vetting;
- Unexpected rate-limit exhaustion or policy restructuring on X's paid tiers;
- Token invalidation cascades across Meta properties triggered by user password rotations;
- Structural schema breakage in Substack's undocumented internal export files;
- Metric false equivalence, particularly when conflating net voting scores with organic impressions.

The recommended implementation strategy follows a modular monolith architecture built on Python (FastAPI), managed PostgreSQL with partitioned time-series snapshot tables, and Redis-backed task workers. The initial implementation must prioritize Meta Graph API and X API v2 to establish the relational and analytics core, ingest Substack via RSS feeds, and initiate LinkedIn developer review early in the development lifecycle.

Deliverable 2: Platform Research Dossiers

1. LinkedIn

Official developer operations for LinkedIn are managed under Microsoft Learn within the

LinkedIn Marketing Solutions framework. Over recent API versions, LinkedIn restructured its developer surface into distinct consumer, marketing, and enterprise categories. The primary integration routes relevant to mTools are the open **Share on LinkedIn** product and the vetted **Community Management API**.

The authentication architecture requires standard OAuth 2.0 using the 3-Legged Authorization Code Flow (response_type=code) to obtain delegated user context. Individual member profiles and LinkedIn Company Pages are both supported, but their functional permissions differ significantly. For open permissions, an application developer can immediately request openid, profile, email, and w_member_social without manual application review. The w_member_social scope authorizes creating, modifying, and deleting member posts, comments, and reactions. However, reading a member's personal posts or retrieving post analytics cannot be accomplished with open permissions. The legacy read scope r_member_social has been decommissioned and remains closed to new applicants.

Accessing personal member analytics requires approval for the **Community Management API**, a vetted product requiring developer application review, business verification, and association with a verified LinkedIn Company Page. Upon approval, the application can request r_member_postAnalytics, which grants access to post performance data. Managing company pages requires w_organization_social, r_organization_social, and r_organization_admin_analysis. OAuth access tokens remain valid for 60 days. Standard applications do not receive automated refresh tokens, meaning members must complete the authorization flow every 60 days unless granted enterprise partner status. Under the Community Management API Development Tier, call volumes are capped at 500 requests per application per 24 hours, and 100 requests per member per 24 hours.

Content retrieval operates through dedicated REST endpoints. Organization posts are retrieved via GET /rest/posts?author=urn:li:organization:{id} using r_organization_soci[span_192](start_span)[span_192](end_span)[span_198](start_span)[span_198](end_span)al. Authenticated member posts and historical reporting data are retrieved through GET /rest/memberCreatorPostAnalytics when authorized with r_member_postAnalytics. The API returns post URNs, canonical URLs, text content, creation timestamps, and media references. However, video uploads to personal member profiles remain restricted via API compared to company pages.

Publishing is fully supported via POST /rest/posts, allowing programmatic distribution of text, images, external web links, and multi-page PDF documents that render as swipeable carousels. Native post scheduling is not supported via the API; all automated scheduling must be orchestrated by the application worker.

The metrics accessible through memberCreatorPostAnalytics provide deep visibility into post performance. The endpoint requires querying metrics via the queryType parameter. Supported metrics include:

LinkedIn Metric Type	API Parameter	Metric Classification	Availability Details
Impressions	IMPRESSION	Direct	Total views of the post entity.
Members Reached	MEMBERS_REACHED	Direct	Unique member accounts viewing the content.
Reactions	REACTION	Direct	Aggregated reaction count.
Comments	COMMENT	Direct	Total top-level

LinkedIn Metric Type	API Parameter	Metric Classification	Availability Details
			comments and replies.
Reshares	RESHARE	Direct	Organic shares across LinkedIn feeds.
Post Saves	POST_SAVE	Direct	Bookmarks recorded on the post.
Post Sends	POST_SEND	Direct	Content shares via 1-on-1 LinkedIn messaging.
Link Clicks	LINK_CLICKS	Direct	Clicks on embedded hyperlinks.
Follower Conversion	FOLLOWER_GAINED_FROM_CONTENT	Direct	New followers attributed to post views.
Profile Views Driven	PROFILE_VIEW_FROM_CONTENT	Direct	Profile traffic driven by the post.
Video Telemetry	Video Analytics API	Direct	Watch time, views, and unique viewers.

When querying post analytics, the aggregation=DAILY parameter cannot be used with MEMBERS_REACHED, LINK_CLICKS, FOLLOWER_GAINED_FROM_CONTENT, or PROFILE_VIEW_FROM_CONTENT; these metrics can only be retrieved as lifetime aggregates using aggregation=TOTAL. Real-time webhooks are restricted to organization social action notifications, requiring mTools to use scheduled polling for member analytics.

2. X (formerly Twitter)

X exposes its developer services via the v2 API hosted at api.x.com. Following changes to developer pricing in 2023, access is divided into four primary tiers: Free, Basic, Pro, and Enterprise.

Authentication uses OAuth 2.0 with Proof Key for Code Exchange (PKCE) via the Authorization Code Flow. Standard accounts, Premium-verified personal accounts, and Verified Organizations share this OAuth architecture. Required scopes include tweet.read, tweet.write, users.read, and offline.access. The offline[span_288](start_span)[span_288](end_span)[span_295](start_span)[span_295](end_span).access scope is critical because X access tokens expire after 2 hours; this scope grants a rotating refresh token that allows the application to maintain access without user re-authentication.

The commercial tier structure dictates functional feasibility:

X Developer Tier	Monthly Fee	Monthly App Read Ceiling	Monthly User Post Limit	Architectural Viability for mTools
Free Tier	$0	~100–500 posts/month	500–1,500 posts/month	**Non-viable**: Insufficient read quota to monitor timelines or retrieve analytics.
Basic Tier	$100	10,000 posts/month	3,000 posts/month	**Minimum Viable**: Supports timeline monitoring and

X Developer Tier	Monthly Fee	Monthly App Read Ceiling	Monthly User Post Limit	Architectural Viability for mTools
				analytics polling for single-user accounts.
Pro Tier	$5,000	1,000,000 posts/month	300,000 posts/month	**Cost-Prohibitive**: Geared toward commercial SaaS aggregators.
Enterprise Tier	Custom	Custom firehose streaming	Custom	**Cost-Prohibitive**: Enterprise contract negotiation.

Content retrieval operates through GET /2/users/:id/tweets, which accepts parameters such as tweet.fields=created_at,public_metrics,non_public_metrics,organic_metrics,attachments,conversation_id. Under the Basic tier, timeline retrieval is restricted to the user's 3,200 most recent tweets via pagination tokens; full-archive search is locked behind the Pro tier. Expansions like expansions=atta[span_123](start_span)[span_123](end_span)[span_132](start_span)[span_132](end_span)[span_141](start_span)[span_141](end_span)chments.media_keys resolve image URLs, video details, and animated GIFs.

Publishing is supported via POST /2/tweets for plain text up to 280 characters (or expanded lengths for Premium accounts), reply threads via reply.in_reply_to_tweet_id, and quote posts. Binary media attachments must be uploaded separately through upload.twitter.com/1.1/media/upload.json before passing the resulting media IDs to the post creation endpoint. Native post scheduling is not supported via the API and must be managed by the application.

Analytics metrics are returned directly within the tweet object fields:

X Metric Type	API Object Field	Metric Classification	Definition / Scope
Impressions	organic_metrics.impression_count	Direct	Total organic views by authenticated user context.
Likes	public_metrics.like_count	Direct	Total public likes.
Retweets	public_metrics.retweet_count	Direct	Public reshares to follower timelines.
Quotes	public_metrics.quote_count	Direct	Shares published with user commentary.
Replies	public_metrics.reply_count	Direct	Direct replies in the post conversation.
Bookmarks	public_metrics.bookmark_count	Direct	Private saves by users.
URL Link Clicks	organic_metrics.url_link_clicks	Direct	Clicks on links within the tweet.
Profile Clicks	organic_metrics.user_profile_clicks	Direct	Visits to the user's profile from the tweet.
Follower Count	User public_metrics.followers_count	Direct	Account-level follower count.

X Metric Type	API Object Field	Metric Classification	Definition / Scope
Engagement Rate	Custom Calculation	Derived	Formula: \frac{\text{likes} + \text{retweet[span_316](start_span)[span_316](end_span)s} + \text{replies} + \text{quotes} + \text{bookmarks}}{\text{impressions}} \times 100.
Deduplicated Reach	Not Reported	Unavailable	X reports gross impressions, not unique reach.

Real-time Account Activity API (AAAPI) webhooks are unavailable on the Free and Basic tiers, making polling necessary. To operate within the Basic tier's 10,000 monthly read limit, mTools must budget polling carefully. A schedule that checks the user timeline every 4 hours consumes approximately 180 requests per month, leaving sufficient capacity for engagement snapshots across active posts.

3. Instagram

Instagram developer integration is managed via the Meta Graph API v26.0. The platform supports two main integration flows: the **Instagram API with Instagram Login** (using graph.instagram.com with instagram_business_* scopes) and the **Instagram API with Unified Login / Facebook Login** (using graph.facebook.com with instagram_* scopes). Meta requires new applications to adopt Unified Login, which removes the historical requirement that an Instagram professional profile be linked to a Facebook Page.

Authentication uses the OAuth 2.0 Authorization Code flow. Supported account types are strictly limited to Professional accounts (Instagram Creator or Instagram Business). Standard personal accounts are unsupported; the legacy Instagram Basic Display API was decommissioned in December 2024. Required scopes include instagram_basic, instagram_manage_insights, instagram_content_publish, and instagram_manage_comments.

In Development Mode, Meta grants Standard Access, allowing immediate testing with up to 50 authorized tester accounts without App Review. Moving an application to Live Mode for multi-tenant use requires Meta App Review and Business Verification. The OAuth exchange converts a short-lived user token (1-hour lifespan) into a 60-day long-lived token via grant_type=ig_exchange_token. This token can be renewed indefinitely before expiration via grant_type=ig_refresh_token without user re-authentication.

Content retrieval is handled through GET /v26.0/{ig-user-id}/media?fields=id,caption,media_type,media_product_type,media_url,thumbnail_url,permalink,timestamp,like_count,comments_count. Supported formats include single images, carousel albums, video Reels, and Stories. Story media and analytics expire after 24 hours unless preserved in profile Highlights.

Publishing requires a three-step asynchronous container flow:

1. Initialize container: POST /v26.0/{ig-user-id}/media passing image_url or video_url.
2. Check container status: GET /v26.0/{creation-id}?fields=status_code until FINISHED is

returned.

3. Publish container: POST /v26.0/{ig-user-id}/media_publish?creation_id={creation-id}.

Native post scheduling is unsupported via the API; calling media_publish publishes content immediately. Comment reading and replying are supported via GET /v26.0/{media-id}/comments and POST /v26.0/{comment-id}/replies.

The Instagram Insights endpoint provides detailed performance data:

Instagram Metric Type	API Parameter	Metric Classification	Definition / Constraints
Impressions	impressions	Direct	Total views of the media object.
Reach	reach	Direct	Deduplicated unique accounts that viewed the content.
Plays / Video Views	plays / views	Direct	Video plays on Reels and video posts.
Likes	likes / total_likes	Direct	Likes count (can include boosted post engagement).
Comments	comments / total_comments	Direct	Total comments on the media object.
Saves	saved	Direct	Number of times accounts saved the media.
Shares	shares	Direct	Shares of the post via Direct Message or Stories.
Profile Interactions	profile_activity	Direct	Breakdown of taps on bio link, call, email, or directions.
Follower Count	User followers_count	Direct	Total professional account followers.
Demographics	Account Insights	Partial	Age, gender, country, and city breakdowns for accounts with \ge 100 followers (retained for 90 days).
Engagement Rate	Custom Calculation	Derived	Formula: \frac{\text{likes} + \text{comments} + \text{shares} + \text{saved}}{\text{impressions}} \times 100 or \frac{\text{total_interactions}}{\text{reach}} \times 100.

Meta Webhooks support real-time events for comments, mentions, and story insights. Post-level metric updates do not fire webhooks, requiring periodic polling. The application-level rate limit provides a generous ceiling based on active users: \text{Calls per hour} = 4,800 \times

\text{Number of Active Users}.

4. Facebook

Facebook integration is provided through the Meta Graph API v26.0 under the Facebook Pages product suite. Personal user profiles cannot be monitored or automated via the API; permissions such as read_stream and user_posts were deprecated for third-party applications to protect user privacy. Access is strictly scoped to Facebook Pages managed by the authenticated user. Authentication uses the OAuth 2.0 Authorization Code flow. Required permissions include:

- pages_show_list: Allows enumerating pages managed by the user via /me/accounts;
- pages_read_engagement: Grants read access to page content, comments, and post metadata;
- pages_manage_posts: Grants authorization to publish, edit, and delete posts on behalf of the page;
- read_insights: Authorizes access to granular post and page analytics.

The token exchange flow converts a short-lived user token into a 60-day long-lived token via grant_type=fb_exchange_token. Querying /me/accounts with this long-lived token yields a **Page Access Token**. Page Access Tokens derived from long-lived user tokens do not expire (expires_in = 0), allowing persistent background synchronization without ongoing re-authentication.
Content retrieval is handled through GET /v26.0/{page-id}/published_posts?fields=id,message,created_time,permalink_url,shares,attachments{media,type,url},reactions.summary(total_count),comments.summary(total_count). Cursor-based pagination enables retrieval of full post histories.
Publishing is supported across media types: text and links use POST /v26.0/{page-id}/feed, photos use POST /{page-id}/photos, and videos use POST /{page-id}/videos. Facebook Pages natively support scheduled posting via the API. By setting published=false[span_163](start_span)[span_163](end_span), scheduled_publish_time={unix_timestamp}, and unpublish[span_164](start_span)[span_164](end_span)ed_content_type=SCHEDULED, content is queued directly on Meta's infrastructure without requiring local scheduler execution. Comment monitoring and responses are supported via GET /{post-id}/comments and P[span_166](start_span)[span_166](end_span)OST /{comment-id}/comments.
Post-level analytics are retrieved through the post insights edge:

Facebook Metric Type	API Edge / Field	Metric Classification	Definition / Scope
Post Impressions	/insights/post_impressions	Direct	Total times the post was viewed.
Unique Reach	/insights/post_impressions_unique	Direct	Deduplicated count of unique users who saw the post.
Engaged Users	/insights/post_engaged_users	Direct	Unique users who clicked, reacted, commented, or shared.
Reactions Summary	reactions.summary(true)	Direct	Reaction counts broken down by LIKE, LOVE, WOW, HAHA, SAD, and ANGRY.

Facebook Metric Type	API Edge / Field	Metric Classification	Definition / Scope
Comments Count	comments.summary(true)	Direct	Total comments on the post.
Post Shares	s[span_484](start_span)[span_484](end_span)[span_491](start_span)[span_491](end_span)hares.count	Direct	Total shares of the post.
Video Views	post_video_views	Direct	Total 3-second or complete video views.
Video Watch Time	post_video_view_time	Direct	Total milliseconds of video playback.
Page Followers / Fans	User followers_count, fan_count	Direct	Total followers and page likes.
Audience Demographics	Page Insights	Partial	Country, city, gender, and age breakdowns for the page audience.
Engagement Rate	Custom Calculation	Derived	Formula: \frac{\text{post_engaged_users}}{\text{post_impressions_unique}} \times 100.

Meta Webhooks support subscriptions to the feed topic for Facebook Pages, delivering real-time payloads when posts, comments, or reactions are created. Time-series metrics (such as impressions over time) must be collected via scheduled polling.

5. Substack

Substack provides no documented, supported public REST API for content retrieval, analytics, or publication management. Official support resources confirm that developer endpoints are not publicly offered. Undocumented internal endpoints (such as those powering substack.com/api/v1) are subject to unannounced changes, protected by bot detection systems, and prohibited by platform terms of service if automated without authorization.
To provide reliable integration while complying with platform policies, mTools implements a hybrid integration architecture:

- **Automated Content Tracking via RSS**: Every Substack publication exposes an open XML RSS 2.0 feed at https://{subdomain}.substack.com/feed (or https://{custom_domain}/feed). The feed provides post titles, canonical URLs, publication timestamps (pubDate), authors (dc:creator), categories, and full HTML body content (content:encoded). This enables automated indexing of published content and publishing consistency tracking. However, the RSS feed provides no private analytics, subscriber counts, or comment data.
- **Private Analytics via Native CSV Export Ingestion**: Substack allows creators to download structured data exports from the publisher dashboard under Settings > Exports and Subscribers > Export. These archives include:
    - posts.csv: Historical post metadata, slugs, publication dates, and aggregate engagement figures;

- subscribers.csv: Subscriber email addresses, subscription tier (free, paid, comp), signup dates, and activity ratings;
- opens.csv / email statistics: Per-post recipient counts, email opens, open rates, and click-through totals.

Automated publishing, draft generation, scheduling, and comment moderation cannot be performed through supported developer channels. Substack must be designated as a manual publishing channel within the mTools calendar, providing draft previews, copy-to-clipboard tools, and manual completion checklists.

Metric availability for Substack is summarized below:

Substack Metric Type	Data Source	Metric Classification	Operational Details
Public Post Metadata	RSS 2.0 XML Feed	Direct	Automated ingestion of titles, slugs, bodies, and dates.
Total Subscribers	subscribers.csv Export	Partial	Ingested from creator export files.
Paid vs Free Subscribers	[span_544](start_span)[span_544](end_span)[span_549](start_span)[span_549](end_span)subscribers.csv Export	Partial	Categorized by subscription tier.
Email Open Rate	opens.csv Export	Partial	Ratio of opens to delivered emails.
Email Click-Through Rate	Email Export Files	Partial	Link clicks within delivered newsletters.
Gross Annualized Revenue	subscribers.csv Export	Partial	Calculated from active paid subscription tiers.
Post Web Views	Dashboard Export	Partial	Web traffic recorded in export files.
Live Metric Updates	Not Supported	Unavailable	Real-time webhooks or polling endpoints do not exist.
Engagement Rate	Custom Calculation	Derived	Formula: \frac{\text{Email Opens}}{\text{Delivered Recipients}} \times 100.

6. Reddit

Reddit exposes developer access through its OAuth 2.0 REST API at oauth.reddit.com. Developer terms enacted in 2023 require application registration and restrict commercial data redistribution.

Authentication supports two grant types under OAuth 2.0: the Authorization Code Flow for multi-user web applications, and the "Script" flow (using client credentials, username, and password) for private administrative tooling. Required scopes include identity (access username and karma), read (access post listings and comment trees), submit (publish posts and comments), and history (retrieve owned submissions and comment histories). Access tokens are valid for 1 hour; requesting duration=permanent during authorization provides a refresh token for ongoing access. Non-commercial use permits up to 100 queries per minute using a

distinct user-agent string.
Content retrieval is handled through GET /user/{username}/submitted for owned posts and GET /user/{username}/comments for discussion history. Listing endpoints support cursor-based pagination via after and before fullname anchors (such as t3_xxxxxx), returning up to 100 items per request and typically covering up to 1,000 historical items. Returned fields include post fullname IDs, titles, body Markdown (selftext), target subreddits, canonical permalinks, net scores, upvote ratios, and comment counts.
Publishing is supported via POST /api/submit, allowing text posts, external links, and media submissions to targeted subreddits (sr={subreddit}) or the user's personal profile (sr=u_{username}). Commenting is supported via POST /api/comment targeting submission fullnames or parent comments. Native post scheduling is not supported via the API and must be managed by the application.
Metric availability on Reddit is subject to platform constraints:

Reddit Metric Type	API Object Field	Metric Classification	Definition / Constraints
Net Score (Karma)	score	Direct	Balance of upvotes minus downvotes (subject to vote fuzzing).
Upvote Ratio	upvote_r[span_577](start_span)[span_577](end_span)atio	Direct	Proportion of positive upvotes (e.g., 0.92).
Comment Count	num_com[span_578](start_span)[span_578](end_span)ments	Direct	Total replies within the submission thread.
Post Impressions / Views	Not Returned	Unavailable	Post Insights are restricted to internal GraphQL queries (gql.reddit.com) and return null via the public REST API.
Outbound Link Clicks	Not Tracked	Unavailable	The API does not track or report external link clicks.
Profile Followers	User Object	Partial	Profile follower counts are accessible via /api/v1/user/{username}/about.
Estimated Gross Upvotes	Custom Calculation	Derived	Heuristic formula: \text{Est. Upvotes} = \frac{\text{Score} \times \text{Upvote Ratio}}{2 \times \text{Upvote Ratio[span_73](start_span)[span_73](end_span)[span_92](start_span)[span_92](end_span)}[span_348](start_span)[

Reddit Metric Type	API Object Field	Metric Classification	Definition / Constraints
			span_348](end_span) - 1} (subject to platform vote fuzzing).

Reddit does not provide outbound webhooks for submission performance or comment notifications. Synchronization relies entirely on polling. Given the 100 requests per minute limit, polling owned submissions every 15 to 30 minutes operates well within rate constraints.

Deliverable 3: Capability Matrix

The master capability matrix below details platform support across authentication, content retrieval, engagement metrics, audience analytics, publishing, and operations.

Capability	LinkedIn	X (Twitter)	Instagram	Facebook	Substack	Reddit	Required Account Type	Required Scopes / Tiers	Endpoint / Source Reference	Known Operational Limitations	Implementation Complexity	Additional Cost	Confidence Level
OAuth 2.0 Connection	Supported	Supported	Supported	Supported	Unsupported	Supported	Personal / Professional (Platform dependent)	Standard OAuth scopes	Official OAuth endpoints	Substack has no OAuth flow; uses RSS/manual upload	Medium	$0	High
Personal Post Retrieval	Partial	Direct	Direct	Unavailable	Direct	Direct	Personal (LinkedIn/X/Reddit), Professional (Instagram)	r_member_postAnalytics (LI), tweet.read (X), instagram_basic (IG), history (Reddit)	/memberCreatorPostAnalytics, /2/users/:id/tweets, /{user}/media, RSS feed, /user/{u}/submitted	Facebook personal profiles deprecated; LinkedIn requires Community Management	High	$100/mo (X Basic)	High

Capability	LinkedIn	X (Twitter)	Instagram	Facebook	Substack	Reddit	Required Account Type	Required Scopes / Tiers	Endpoint / Source Reference	Known Operational Limitations	Implementation Complexity	Additional Cost	Confidence Level
										review			
Org / Page Post Retrieval	Direct	Direct	Direct	Direct	Unsupported	Unavailable	Business / Organization / Page	r_organization_social (LI), pages_read_engagement (FB)	/rest/posts (LI), /{page-id}/publi[span_106](start_span)[span_106](end_span)[span_108](start_span)[span_108](end_span)[span_110](start_span)[span_110](end_span)[span_112](start_span)[span_112](end_span)shed_posts (FB)	Substack/Reddit do not support organization pages	Low	$0	High
Historical	Partial	Partial	Direct	Direct	Partial	Partial	Any suppo	Standard	Paginated	X Basic	Medium	$0	High

Capability	LinkedIn	X (Twitter)	Instagram	Facebook	Substack	Reddit	Required Account Type	Required Scopes / Tiers	Endpoint / Source Reference	Known Operational Limitations	Implementation Complexity	Additional Cost	Confidence Level
Post Retrieval							rted	read scopes	list endpoints	limited to 3,200 recent tweets; Substack RSS limited to ~20–50 items; Reddit capped at ~1,000			
Text & Media Metadata	Direct	Direct	Direct	Direct	Direct	Direct	Any supported	Standard read scopes	Object detail fields	Video metadata on LinkedIn member posts is constrained	Low	$0	High
Impressions	Direct	Direct	Direct	Direct	Partial	Unavailable	Professional / Business / Creator / User (X)	r_member_postAnalytics (LI), tweet.read (X), instagram_manage	Analytics endpoints	Excluded from Reddit public REST API; Substack requires	High	$100/mo (X Basic)	High

Capability	LinkedIn	X (Twitter)	Instagram	Facebook	Substack	Reddit	Required Account Type	Required Scopes / Tiers	Endpoint / Source Reference	Known Operational Limitations	Implementation Complexity	Additional Cost	Confidence Level
								_insights (IG), read_insights (FB)		manual CSV upload			
Reach (Unique)	Direct	Unavailable	Direct	Direct	Partial	Unavailable	Professional / Page / Creator	Insights scopes	MEMBERS_REACHED (LI), reach (IG/FB)	X does not provide unique reach metrics	Medium	$0	High
Views / Plays	Direct	Unavailable	Direct	Direct	Partial	Unavailable	Professional / Business	Media read & insights scopes	Video insights endpoints	Instagram reports plays for Reels	Medium	$0	High
Likes / Reactions	Direct	Direct	Direct	Direct	Partial	Direct (Net)	Any supported	Standard read scopes	like_count, reactions.summary, score	Reddit exposes net score, not gross upvotes; Substack captures via CSV	Low	$0	High
Comments / Replies	Direct	Direct	Direct	Direct	Unavailable	Direct	Any supported	Standard read scopes	Post detail objects	Substack comments are	Low	$0	High

Capability	LinkedIn	X (Twitter)	Instagram	Facebook	Substack	Reddit	Required Account Type	Required Scopes / Tiers	Endpoint / Source Reference	Known Operational Limitations	Implementation Complexity	Additional Cost	Confidence Level
										closed without API access			
Shares / Reposts	Direct	Direct	Direct	Direct	Partial	Unavailable	Any supported	Read scopes	Post detail objects	Reddit cross-posting is not tracked as a metric	Low	$0	High
Saves / Bookmarks	Direct	Direct	Direct	Unavailable	Unavailable	Unavailable	Professional / User (LI/X/IG)	Read / Insights scopes	POST_SAVE (LI), bookmark_cou[span_460](start_span)[span_460](end_span)[span_465](start_span)[span_465](end_span)nt (X), saved (IG)	Facebook, Substack, Reddit do not expose bookmark metrics	Low	$0	High
Link Clicks	Direct	Direct	Partial	Partial	Partial	Unavailable	Professional / User (LI/X)	Insights / Organic	LINK_CLICKS (LI), organi	Instagram only tracks	Medium	$0	High

Capability	LinkedIn	X (Twitter)	Instagram	Facebook	Substack	Reddit	Required Account Type	Required Scopes / Tiers	Endpoint / Source Reference	Known Operational Limitations	Implementation Complexity	Additional Cost	Confidence Level
								metrics	c_metrics.url_link_clicks (X)	bio link clicks; Substack requires CSV			
Follower Growth	Direct	Direct	Direct	Direct	Partial	Direct	Any supported	Identity / Read scopes	User / Page profile objects	Substack captured via CSV subscriber history	Low	$0	High
Subscriber Growth	Unavailable	Unavailable	Unavailable	Unavailable	Partial	Unavailable	Substack Creator	Manual CSV Export	subscribers.csv ingestion	Non-Substack platforms do not support paid newsletters	Medium	$0	High
Audience Demographics	Direct	Unavailable	Partial	Partial	Partial	Unavailable	Professional / Page	Insights permissions	Demographics endpoints	Instagram requires \ge 100 followers; X demographics require Enterprise	High	$0	High
Post	Direct	Direct	Direct	Direct	Unava	Direct	Any	Write	Post	Subst	Mediu	$0	High

Capability	LinkedIn	X (Twitter)	Instagram	Facebook	Substack	Reddit	Required Account Type	Required Scopes / Tiers	Endpoint / Source Reference	Known Operational Limitations	Implementation Complexity	Additional Cost	Confidence Level
Publishing					ilable		supported	scopes (w_member_social, etc.)	creation endpoints	ack requires manual authoring; Instagram requires async container flow	m		
Post Scheduling	App-side	App-side	App-side	Direct	Unavailable	App-side	Page (Facebook natively)	Publishing scopes	scheduled_publish_time (Facebook only)	Only Facebook natively supports API scheduling; others require mTools queue	Medium	$0	High
Comment Management	Direct	Direct	Direct	Direct	Unavailable	Direct	Any supported	Comment moderation scopes	Comment endpoints	Substack unsupported; X requires Basic tier write quota	Medium	$0	High

Capability	LinkedIn	X (Twitter)	Instagram	Facebook	Substack	Reddit	Required Account Type	Required Scopes / Tiers	Endpoint / Source Reference	Known Operational Limitations	Implementation Complexity	Additional Cost	Confidence Level
Real-Time Webhooks	Partial	Unavailable	Direct	Direct	Unavailable	Unavailable	Professional / Page	Webhook subscriptions	Webhook callback verification	X Account Activity API locked behind Pro tier ($5,000/mo); others require polling	High	$0	High
Historical Analytics	Direct	Partial	Direct (90d)	Direct	Partial	Unavailable	Professional / Business	Reporting scopes	Lifetime insights endpoints	Instagram limited to 90 days; X Basic limited to recent posts; Reddit has no historical stats	Medium	$0	High
AI Content Insights	Inferred	Inferred	Inferred	Inferred	Inferred	Inferred	Internal mTools processing	None (Internal LLM pipeline)	LLM inference layer	Derived from normalized telemetry; no direct platform	Medium	LLM API tokens	High

Capability	LinkedIn	X (Twitter)	Instagram	Facebook	Substack	Reddit	Required Account Type	Required Scopes / Tiers	Endpoint / Source Reference	Known Operational Limitations dependency	Implementation Complexity	Additional Cost	Confidence Level

---

Deliverable 4: Architecture Decision Record (ADR)

ADR-001: Backend Framework and Language Selection

The backend must handle concurrent REST polling, parse RSS feeds and CSV files, process incoming webhooks, normalize varied JSON payloads, and orchestrate LLM analytical pipelines.

Python 3.12+ with FastAPI is selected as the primary backend runtime. Python offers a mature ecosystem for data manipulation and generative AI integration, including Pydantic, Instructor, Polars, and official provider SDKs. FastAPI provides asynchronous ASGI performance (asyncio) capable of handling incoming webhooks and concurrent outgoing REST requests.

TypeScript with Node.js (NestJS) was evaluated and rejected because its ecosystem lacks comparable data manipulation and AI extraction libraries. Go was also rejected due to the development overhead required for dynamic JSON reshaping and the lack of mature structured output libraries for generative AI pipelines.

ADR-002: Primary Datastore and Snapshot Time-Series Modeling

Social media engagement metrics change continuously over the lifespan of a post. The system must retain immutable time-series snapshots to track historical trends while accommodating platform-specific metadata.

PostgreSQL 16+ with declarative monthly table partitioning for metric snapshot tables is selected as the primary datastore. PostgreSQL provides relational integrity for user identity, account integrations, and post normalization, while offering native JSONB support for unstructured platform payloads. Partitioning the snapshot tables ensures query performance remains consistent as snapshot records accumulate over time.

MongoDB was rejected because it lacks the relational integrity needed for token isolation, audit logging, and SQL-based aggregation queries. A dedicated time-series database such as InfluxDB was rejected because it introduces operational overhead and synchronization lag between the post entities and their associated metric series.

ADR-003: Ingestion Strategy and Rate-Limit Budgeting Engine

Data collection patterns across the six platforms range from real-time webhooks on Meta properties to strict polling limits on X (10,000 reads/month on Basic) and LinkedIn (100 calls/member/day).

An asynchronous worker engine with tiered exponential polling backoff, implemented using

Redis and ARQ, is selected for data synchronization. Ingestion schedules are tiered based on post age:

- Posts under 48 hours old are polled every 4 hours to capture active engagement velocity.
- Posts between 3 and 14 days old are polled once every 24 hours.
- Posts between 15 and 30 days old are polled once weekly, after which they are marked static.
- Webhook events on Meta platforms bypass polling schedules and trigger targeted single-object reconciliation jobs immediately.

Synchronous, request-time polling on user dashboard loads was rejected because it introduces page latency, risks immediate rate-limit exhaustion, and prevents automated historical snapshot logging.

ADR-004: Multi-Platform Schema Normalization Strategy

The database must normalize post objects across platforms for unified filtering while preserving platform-specific data like Reddit upvote ratios, LinkedIn message sends, and X bookmarks.
A hybrid relational and document model is selected, combining core normalized relational columns with a platform_raw_data JSONB extension column. Standard attributes (post_id, published_at, canonical_url, content_text) are stored in typed columns to enable cross-platform sorting and search. Platform-specific metrics are preserved in platform_raw_data and queried using PostgreSQL JSON operators for detailed views.
A lowest-common-denominator schema was rejected because it discards valuable platform metrics. Creating distinct polymorphic tables for each platform was also rejected because it complicates cross-platform queries and dashboard views.

ADR-005: AI Analysis Pipeline and Prompt Injection Mitigation

User-generated social content (including posts, comments, and replies) represents untrusted external input. Passing raw social text directly into LLM prompts creates vulnerabilities to prompt injection and instruction override attacks.
A deterministic analytical pipeline with context isolation and structured output validation (using Pydantic and Instructor) is selected for the AI layer. Numerical aggregations, trend detections, and performance rankings are computed deterministically in Python before invoking the LLM. Untrusted text strings (such as comment bodies) are stripped of control characters, truncated, and wrapped in explicit XML data delimiters (<untrusted_content>). System prompts instruct the model to treat content within these tags as passive data, and model outputs are validated against Pydantic schemas before persistence.
Allowing the LLM to compute performance metrics directly from raw data was rejected due to token costs and the risk of mathematical hallucinations.

Deliverable 5: System Architecture

The mTools system architecture separates platform-specific connector logic from core application workflows, maintaining a normalized data pipeline and consistent security controls.

Subsystem Architecture

The system is organized into six primary architectural layers:

- **Presentation Layer**: A Next.js 14 single-page application rendering overview dashboards, metric charts, content libraries, and publishing workflows.
- **API Gateway & Application Layer**: A FastAPI application providing REST API endpoints, enforcing authentication, managing tenant boundaries, and handling webhook routing.
- **Connector & Adapter Layer**: Platform-specific adapter modules implementing a shared SocialPlatformAdapter interface. Adapters manage OAuth handshakes, token refreshes, rate-limit tracking, error translation, and payload normalization.
- **Task Queue & Ingestion Workers**: A Redis-backed asynchronous worker system (ARQ) executing scheduled synchronization jobs, processing webhook events, and managing retry queues.
- **Persistence Layer**: A PostgreSQL 16 database storing user profiles, encrypted credentials, normalized post entities, audit logs, and partitioned snapshot tables.
- **AI Intelligence Engine**: An analysis service computing statistical summaries, structuring sanitized prompt contexts, and querying external LLMs for qualitative insights.

Interaction Flows and Data Pipelines

The system coordinates data across five primary workflows:

1. OAuth Authentication Flow

When a user connects an account, the frontend submits a connection request to POST /api/v1/integrations/{platform}/connect. The API gateway invokes the matching platform adapter to generate a state token, CSRF secret, and the platform authorization URL. The user completes authentication on the provider's domain and is redirected to the callback endpoint with an authorization code. The platform adapter exchanges the code server-side for access and refresh tokens. Tokens are encrypted using AES-256-GCM and stored in the database, and an initial account discovery task is enqueued in Redis.

2. Content Ingestion and Snapshot Flow

Scheduled worker jobs initiate periodic synchronization based on the rate-limit budget stored in Redis. The worker calls the adapter's fetch_recent_posts method, and the returned payloads are normalized into uniform post entities. Posts are written to the database using idempotent upserts (ON CONFLICT DO UPDATE). The worker then retrieves current engagement telemetry, writes an immutable snapshot row to post_metrics_snapshots marked with provenance platform_direct, and calculates derived engagement ratios.

3. Webhook Ingestion Flow

Meta platforms deliver real-time notifications to /api/v1/webhooks/{platform}. The gateway validates the signature header (such as X-Hub-Signature-256) against the stored platform secret. The event payload is recorded in webhook_events with status RECEIVED, an asynchronous task is enqueued in Redis, and a 200 OK response is returned within the provider's timeout window. The background worker parses the event, retrieves updated entity

details via the adapter, and updates the database.

4. AI Insights Generation Flow

The analysis pipeline runs on a scheduled weekly cadence or on manual request. Deterministic SQL queries calculate performance averages, identify engagement anomalies, and categorize top-performing content formats across historical snapshots. The analysis service formats these findings, places untrusted post text within XML isolation tags, and submits the payload to the LLM. The model returns structured observations, supporting post references, confidence ratings, and recommendations, which are validated against Pydantic schemas and written to ai_insights.

5. Failure Recovery and Dead-Letter Flow

Transient external errors (such as HTTP 429 rate limits or 503 service disruptions) trigger exponential backoff retries (2^n \times \text{base_delay}). When an authentication error occurs (HTTP 401 or 403), the adapter attempts an automated token refresh. If token refresh fails due to revoked permissions, the integration status is set to NEEDS_REAUTH, and an in-app alert is generated. Tasks that exceed maximum retry thresholds are moved to a Redis dead-letter queue, and an entry is logged in audit_logs for administrative review.

Deliverable 6: Database Design

The PostgreSQL database schema below defines table structures, constraints, indexes, and partition schemes for mTools.

```
-- mTools Core Relational Database Schema (PostgreSQL 16+)
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- Custom Domain Enumerations
CREATE TYPE platform_type AS ENUM ('linkedin', 'x', 'instagram',
'facebook', 'substack', 'reddit');
CREATE TYPE integration_status AS ENUM ('active', 'expired',
'revoked', 'rate_limited', 'error');
CREATE TYPE post_status AS ENUM ('published', 'scheduled', 'draft',
'failed', 'archived');
CREATE TYPE media_type AS
ENU[span_447](start_span)[span_447](end_span)[span_456](start_span)[sp
an_456](end_span)M ('image', 'video', 'carousel', 'document',
'audio');
CREATE TYPE metric_provenance AS ENUM ('platform_direct',
'calculated', 'manual_import', 'ai_inferred');
CREATE TYPE insight_confidence AS ENUM ('high', 'medium', 'low');

-- 1. User Management
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
    email VARCHAR(255) UNIQUE NOT NULL,
    hashed_password VARCHAR(255) NOT NULL,
    full_name VARCHAR(100),
    timezone VARCHAR(50) DEFAULT 'UTC' NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL,
    updated_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

-- 2. Platform Integrations Configuration
CREATE TABLE platform_integrations (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    platform platform_type NOT NULL,
    status integration_status DEFAULT 'active' NOT NULL,
    external_account_id VARCHAR(255) NOT NULL,
    account_name VARCHAR(255) NOT NULL,
    account_type VARCHAR(50) NOT NULL, -- e.g., 'personal',
'organization', 'creator', 'page'
    profile_url VARCHAR(500),
    avatar_url VARCHAR(500),
    sync_frequency_minutes INT DEFAULT 240 NOT NULL,
    last_synced_at TIMESTAMPTZ,
    error_detail TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL,
    updated_at TIMESTAMPTZ DEFAULT NOW() NOT NULL,
    CONSTRAINT uq_user_platform_account UNIQUE (user_id, platform,
external_account_id)
);

-- 3. Encrypted OAuth Credentials
CREATE TABLE oauth_credentials (
    id UUID PRIMARY KEY DEFAULT
uui[span_413](start_span)[span_413](end_span)[span_421](start_span)[sp
an_421](end_span)d_generate_v4(),
    integration_id UUID NOT NULL REFERENCES platform_integrations(id)
ON DELETE CASCADE UNIQUE,
    encrypted_access_token TEXT NOT NULL,
    encrypted_refresh_token TEXT,
    token_type VARCHAR(50) DEFAULT 'Bearer',
    scopes TEXT[] NOT NULL,
    expires_at TIMESTAMPTZ,
    refresh_token_expires_at TIMESTAMPTZ,
    encryption_key_id VARCHAR(50) NOT NULL,
    updated_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

-- 4. Content Topics & Taxonomy
CREATE TABLE content_topics (
```

```
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    name VARCHAR(100) NOT NULL,
    color_hex VARCHAR(7) DEFAULT '#6B7280',
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL,
    CONSTRAINT uq_user_topic UNIQUE (user_id, name)
);

-- 5. Normalized Posts
CREATE TABLE posts (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    integration_id UUID NOT NULL REFERENCES platform_integrations(id)
ON DELETE CASCADE,
    external_post_id VARCHAR(255) NOT NULL,
    canonical_url VARCHAR(1000) NOT NULL,
    status post_status DEFAULT 'published' NOT NULL,
    published_at TIMESTAMPTZ NOT NULL,
    content_text TEXT,
    content_title VARCHAR(500), -- Substack article titles, Reddit
submission titles
    author_name VARCHAR(255),
    language VARCHAR(10),
    cross_post_group_id UUID,
    platform_raw_data JSONB NOT NULL DEFAULT '{}'::jsonb,
    is_deleted_on_platform BOOLEAN DEFAULT FALSE NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL,
    updated_at TIMESTAMPTZ DEFAULT NOW() NOT NULL,
    CONSTRAINT uq_integration_external_post UNIQUE (integration_id,
external_post_id)
);

CREATE INDEX idx_posts_published_at ON posts(published_at DESC);
CREATE INDEX idx_posts_integration_published ON posts(integration_id,
published_at DESC);
CREATE INDEX idx_posts_cross_post_group ON posts(cross_post_group_id)
WHERE cross_post_group_id IS NOT NULL;
CREATE INDEX idx_posts_raw_data_gin ON posts USING GIN
(platform_raw_data);

-- 6. Post Media Attachments
CREATE TABLE post_media (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    post_id UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
    media_type media_type NOT NULL,
    media_url TEXT NOT NULL,
    thumbnail_url TEXT,
    media_order INT DEFAULT 0 NOT NULL,
    duration_seconds INT,
```

```
    width INT,
    height INT,
    platform_media_id VARCHAR(255),
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

-- 7. Post-to-Topic Junction
CREATE TABLE post_topic_mappings (
    post_id UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
    topic_id UUID NOT NULL REFERENCES content_topics(id) ON DELETE
CASCADE,
    PRIMARY KEY (post_id, topic_id)
);

-- 8. Post Metrics Snapshots (Partitioned Time-Series)
CREATE TABLE post_metrics_snapshots (
    id UUID DEFAULT uuid_generate_v4(),
    post_id UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
    captured_at TIMESTAMPTZ NOT NULL,
    provenance metric_provenance DEFAULT 'platform_direct' NOT NULL,

    -- Normalized Metrics
    impressions BIGINT,
    reach BIGINT,
    views BIGINT,
    likes BIGINT,
    comments BIGINT,
    shares BIGINT,
    saves BIGINT,
    clicks BIGINT,

    -- Calculated Metrics
    engagement_rate NUMERIC(6, 4),

    -- Platform-Specific Extensions
    extended_metrics JSONB DEFAULT '{}'::jsonb NOT NULL,

    PRIMARY KEY (id, captured_at)
) PARTITION BY RANGE (captured_at);

-- Partition Tables for 2026/2027 Quarters
CREATE TABLE post_metrics_snapshots_2026_q1 PARTITION OF
post_metrics_snapshots
    FOR VALUES FROM ('2026-01-01 00:00:00+00') TO ('2026-04-01
00:00:00+00');
CREATE TABLE post_metrics_snapshots_2026_q2 PARTITION OF
post_metrics_snapshots
    FOR VALUES FROM ('2026-04-01 00:00:00+00') TO ('2026-07-01
```

```
00:00:00+00');
CREATE TABLE post_metrics_snapshots_2026_q3 PARTITION OF
post_metrics_snapshots
    FOR VALUES FROM ('2026-07-01 00:00:00+00') TO ('2026-10-01
00:00:00+00');
CREATE TABLE post_metrics_snapshots_2026_q4 PARTITION OF
post_metrics_snapshots
    FOR VALUES FROM ('2026-10-01 00:00:00+00') TO ('2027-01-01
00:00:00+00');

CREATE INDEX idx_snapshots_post_captured ON
post_metrics_snapshots(post_id, captured_at DESC);

-- 9. Account-Level Metrics Snapshots
CREATE TABLE account_metrics_snapshots (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    integration_id UUID NOT NULL REFERENCES platform_integrations(id)
ON DELETE CASCADE,
    captured_at TIMESTAMPTZ NOT NULL,
    follower_count BIGINT,
    following_count BIGINT,
    subscriber_count BIGINT, -- Substack specific
    profile_views BIGINT,
    raw_demographics JSONB DEFAULT '{}'::jsonb NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

CREATE INDEX idx_account_snapshots_integration_captured ON
account_metrics_snapshots(integration_id, captured_at DESC);

-- 10. Post Comments & Engagement
CREATE TABLE comments (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    post_id UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
    external_comment_id VARCHAR(255) NOT NULL,
    parent_comment_id VARCHAR(255),
    author_name VARCHAR(255) NOT NULL,
    author_avatar_url VARCHAR(500),
    content_text TEXT NOT NULL,
    like_count INT DEFAULT 0,
    published_at TIMESTAMPTZ NOT NULL,
    sentiment_score NUMERIC(4, 3),
    is_requires_attention BOOLEAN DEFAULT FALSE NOT NULL,
    platform_raw_data JSONB DEFAULT '{}'::jsonb NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL,
    CONSTRAINT uq_post_external_comment UNIQUE (post_id,
external_comment_id)
);
```

```
CREATE INDEX idx_comments_post_published ON comments(post_id,
published_at DESC);

-- 11. Publishing Calendar & Scheduled Executions
CREATE TABLE scheduled_publications (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    scheduled_at TIMESTAMPTZ NOT NULL,
    status post_status DEFAULT 'scheduled' NOT NULL,
    target_platforms platform_type[] NOT NULL,
    master_content_text TEXT NOT NULL,
    platform_customizations JSONB DEFAULT '{}'::jsonb NOT NULL,
    media_attachment_urls TEXT[],
    published_post_ids UUID[],
    execution_error TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL,
    updated_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

CREATE INDEX idx_scheduled_pub_status_time ON
scheduled_publications(status, scheduled_at ASC);

-- 12. Synchronization Jobs Ledger
CREATE TABLE sync_jobs (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    integration_id UUID NOT NULL REFERENCES platform_integrations(id)
ON DELETE CASCADE,
    job_type VARCHAR(50) NOT NULL,
    status VARCHAR(50) NOT NULL,
    started_at TIMESTAMPTZ NOT NULL,
    completed_at TIMESTAMPTZ,
    records_synced INT DEFAULT 0,
    error_message TEXT
);

-- 13. Webhook Events Ledger
CREATE TABLE webhook_events (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    platform platform_type NOT NULL,
    event_type VARCHAR(100) NOT NULL,
    payload JSONB NOT NULL,
    signature_header VARCHAR(500),
    is_processed BOOLEAN DEFAULT FALSE NOT NULL,
    processed_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);
```

```
-- 14. AI Insights & Evidence Ledger
CREATE TABLE ai_insights (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    insight_type VARCHAR(50) NOT NULL,
    observation TEXT NOT NULL,
    recommendation TEXT NOT NULL,
    supporting_evidence JSONB NOT NULL,
    data_limitations TEXT,
    confidence insight_confidence NOT NULL,
    analysis_window_start TIMESTAMPTZ NOT NULL,
    analysis_window_end TIMESTAMPTZ NOT NULL,
    is_dismissed BOOLEAN DEFAULT FALSE NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

-- 15. System Notifications
CREATE TABLE notifications (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    notification_type VARCHAR(50) NOT NULL,
    title VARCHAR(255) NOT NULL,
    message TEXT NOT NULL,
    metadata JSONB DEFAULT '{}'::jsonb NOT NULL,
    is_read BOOLEAN DEFAULT FALSE NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

-- 16. Security & Operational Audit Log
CREATE TABLE audit_logs (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    action VARCHAR(100) NOT NULL,
    entity_type VARCHAR(50) NOT NULL,
    entity_id VARCHAR(255),
    ip_address INET,
    details JSONB DEFAULT '{}'::jsonb NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW() NOT NULL
);
```

Snapshot Retention and Lifecycle Policy

To maintain query responsiveness as time-series data grows, post metrics are managed under a three-tiered lifecycle:

- **Active Tier (0–7 days post-publication)**: Metric snapshots are collected every 4 hours and stored in high resolution within the active quarterly partition.
- **Consolidated Daily Tier (8–90 days post-publication)**: Intra-day records are rolled up

daily at 23:59:59Z, and 4-hour intermediate snapshots are pruned.
- **Archival Tier (> 90 days post-publication)**: Snapshots are rolled up weekly. Partition tables older than 365 days are exported to compressed Parquet files in cold storage, leaving final cumulative metrics stored on the primary posts record.

Deliverable 7: API and Integration Contracts

Internal APIs isolate presentation logic from the heterogeneous platform adapters. All endpoints conform to standard REST conventions, utilize snake_case JSON payloads, and return uniform error structures.

1. Connect Account

- **Endpoint**: POST /api/v1/integrations/{platform}/connect
- **Purpose**: Generates state, stores OAuth session secrets, and returns authorization URL.
- **Request Payload**:

```
{
  "redirect_uri":
"https://mtools.app/api/v1/integrations/linkedin/callback",
  "account_type": "member"
}
```

- **Response Payload (200 OK)**:

```
{
  "authorization_url":
"https://www.linkedin.com/oauth/v2/authorization?response_type=code&cl
ient_id=77xxxxxx&redirect_uri=https%3A%2F%2Fmtools.app%2Fapi%2Fv1%2Fin
tegrations%2Flinkedin%2Fcallback&scope=openid%20profile%20email%20w_me
mber_social%20r_member_postAnalytics&state=d93j20dkslc02847",
  "state": "d93j20dkslc02847",
  "expires_in_seconds": 600
}
```

2. List Connected Accounts & Integration Health

- **Endpoint**: GET /api/v1/integrations
- **Response Payload (200 OK)**:

```
{
  "integrations": [
    {
      "id": "e4a7a8d0-6f91-4cf5-9988-cb94e5e40e21",
      "platform": "linkedin",
      "account_name": "Jane Doe",
      "account_type": "member",
      "status": "active",
      "last_synced_at": "2026-10-26T04:12:00Z",
```

```
      "token_expires_at": "2026-12-15T00:00:00Z",
      "rate_limit_usage": {
        "used_today": 42,
        "daily_limit": 100,
        "resets_at": "2026-10-27T00:00:00Z"
      }
    },
    {
      "id": "c1f3b2e5-4a11-4798-bce3-4411aa880912",
      "platform": "x",
      "account_name": "@janedoe_tech",
      "account_type": "personal",
      "status": "active",
      "last_synced_at": "2026-10-26T02:00:00Z",
      "token_expires_at": "2026-10-26T06:00:00Z",
      "rate_limit_usage": {
        "used_this_month": 1420,
        "monthly_limit": 10000,
        "resets_at": "2026-11-01T00:00:00Z"
      }
    }
  ]
}
```


3. Unified Content Library Query

- **Endpoint**: GET /api/v1/posts
- **Query Parameters**: platforms=linkedin,x,instagram, date_from=2026-10-01, sort_by=impressions, order=desc, limit=20
- **Response Payload (200 OK)**:


```
{
  "data": [
    {
      "id": "99b0c741-2a21-4fce-b248-18e38d9fbb34",
      "platform": "linkedin",
      "canonical_url":
"https://www.linkedin.com/feed/update/urn:li:share:7123456789",
      "published_at": "2026-10-20T14:30:00Z",
      "content_text": "Evaluating multi-region database replication
patterns...",
      "media": [
        {
          "type": "document",
          "url": "https://storage.mtools.app/media/arch-diagram.pdf",
          "thumbnail_url":
"https://storage.mtools.app/thumbnails/arch-diagram.jpg"
        }
```

```
      ],
      "metrics": {
        "provenance": "platform_direct",
        "captured_at": "2026-10-26T04:00:00Z",
        "impressions": 14500,
        "reach": 9800,
        "likes": 342,
        "comments": 48,
        "shares": 29,
        "saves": 115,
        "clicks": 412,
        "engagement_rate": 0.0652
      },
      "platform_specific": {
        "post_sends": 18,
        "profile_views_from_content": 45
      }
    }
  ],
  "pagination": {
    "total_records": 1,
    "limit": 20,
    "has_more": false
  }
}
```


4. Normalized Analytics Aggregate

- **Endpoint**: GET /api/v1/analytics/overview?window=30d
- **Response Payload (200 OK)**:


```
{
  "window": {
    "start": "2026-09-26T00:00:00Z",
    "end": "2026-10-26T00:00:00Z"
  },
  "summary": {
    "total_posts_published": 28,
    "aggregate_impressions": {
      "value": 142050,
      "participating_platforms": ["linkedin", "x", "instagram",
"facebook"],
      "excluded_platforms": ["reddit", "substack"],
      "caveat": "Reddit excludes impressions; Substack metrics missing
latest CSV import."
    },
    "aggregate_reactions": {
      "value": 5420,
```

```
      "participating_platforms": ["linkedin", "x", "instagram",
"facebook", "reddit"]
    },
    "aggregate_comments": {
      "value": 892,
      "participating_platforms": ["linkedin", "x", "instagram",
"facebook", "reddit"]
    }
  },
  "platform_breakdown": {
    "linkedin": { "posts": 8, "impressions": 68000, "interactions":
2450 },
    "x": { "posts": 14, "impressions": 45200, "interactions": 1210 },
    "reddit": { "posts": 6, "impressions": null, "interactions": 1760
}
  }
}
```


5. AI Insights & Evidence Retrieval

- **Endpoint**: GET /api/v1/insights?status=active
- **Response Payload (200 OK)**:


```
{
  "insights": [
    {
      "id": "7bf31742-b0cc-499e-97c2-f191d90c9b4e",
      "insight_type": "format_correlation",
      "observation": "Technical architecture PDF carousels on LinkedIn
achieve 3.2x higher post saves and 2.1x higher reach compared to plain
text updates.",
      "evidence": {
        "sample_size_posts": 12,
        "carousel_avg_saves": 112,
        "text_avg_saves": 35,
        "sample_post_ids": ["99b0c741-2a21-4fce-b248-18e38d9fbb34"]
      },
      "recommendation": "Package upcoming deep dives on distributed
consensus into downloadable multi-page PDF documents rather than
standard text posts.",
      "data_limitations": "Evaluated across 12 posts over a 60-day
window; findings reflect correlation, not platform algorithmic
causation.",
      "confidence": "high",
      "created_at": "2026-10-25T08:00:00Z"
    }
  ]
}
```

6. Scheduled Publication Management

- **Endpoint**: POST /api/v1/publishing/schedule
- **Request Payload**:

```
{
  "scheduled_at": "2026-10-28T14:00:00Z",
  "target_platforms": ["linkedin", "facebook", "substack"],
  "master_content_text": "Announcing the release of our open-source
distributed cache...",
  "platform_customizations": {
    "linkedin": {
      "commentary": "Announcing our new distributed cache! Read the
whitepaper attached. #opensource #distributedsystems"
    },
    "facebook": {
      "use_native_scheduler": true
    },
    "substack": {
      "manual_execution_required": true,
      "email_subject": "Announcing our new open-source distributed
cache"
    }
  },
  "media_urls": ["https://storage.mtools.app/media/whitepaper.pdf"]
}
```

- **Response Payload (201 Created)**:

```
{
  "schedule_id": "3c98b671-0f84-48f8-b391-7663d2745a90",
  "status": "scheduled",
  "execution_plan": {
    "linkedin": "mtools_worker_dispatch",
    "facebook": "native_platform_staged",
    "substack": "manual_action_reminder"
  }
}
```

Deliverable 8: UI and Dashboard Specification

The user interface is designed to present dense quantitative analytics alongside qualitative AI insights, with explicit visual signaling when metrics cannot be cleanly aggregated across platforms.

Core Dashboard Views

1. Overview Dashboard

The Overview Dashboard acts as the primary executive monitoring surface. The top ribbon renders four summary cards: Total Publications, Aggregate Impressions, Total Interactions (Reactions, Comments, and Shares), and Net Audience Growth.
When displaying metrics with missing platform data—such as aggregate impressions, which cannot include Reddit or Substack—an informational warning badge is displayed adjacent to the metric value. Clicking the badge reveals an explanation documenting platform data limitations.
Below the ribbon, a dual-axis time-series chart renders publication volume on the primary axis against total engagement interactions on the secondary axis across 7-day, 30-day, and 90-day windows. A stacked bar chart displays the distribution of published content across platforms alongside the corresponding share of total engagement generated.

2. Content Library

The Content Library provides a searchable table of all published content across connected platforms. Users can filter by platform, date range, media format (text, single image, carousel/document, video), assigned topic tags, and performance quartiles.
The table displays publication timestamps (rendered in local time with UTC tooltips), platform indicators, post snippets, media format badges, and standard metrics: impressions, reach, likes, comments, shares, saves, link clicks, and calculated engagement rates. When a platform does not support a given metric—such as reach on X or impressions on Reddit—the table cell renders an em-dash (—) rather than zero, with a hover tooltip explaining that the platform API does not supply the data point.

3. Post Detail View

The Post Detail View displays detailed telemetry for a single publication. The top panel presents the canonical URL, post author, published timestamp, full post copy, and media attachments. Below the metadata, an engagement velocity chart plots metric snapshots over the first 72 hours post-publication, illustrating early performance trajectories. A data provenance panel specifies the timestamp of the last API sync, indicates whether metrics reflect live polling or manual CSV imports, and exposes the raw JSON payload for debugging. An engagement tab renders threaded comments alongside basic sentiment scores.

4. Publishing Calendar

The Publishing Calendar displays scheduled and published content across a weekly or monthly grid. Each scheduled entry displays visual indicators distinguishing automated publishing channels from manual ones:

- Posts scheduled for Facebook, Instagram, LinkedIn, X, or Reddit show an automated badge.
- Posts targeting Substack show a manual reminder badge. When the scheduled time arrives, the interface surfaces a publishing checklist with copy-to-clipboard buttons for newsletter text and attachments, along with a "Mark as Published" action.

Deliverable 9: Implementation Roadmap

The implementation roadmap divides development into seven sequential milestones to resolve platform access prerequisites and commercial terms before core application engineering begins.

Milestone 0: Feasibility, Developer Access & Commercial Validation

- **Deliverables**: Provision developer accounts, configure developer apps, submit access requests, and verify billing tiers.
    - Submit LinkedIn Community Management API application with company page verification.
    - Create Meta Developer App, configure Unified Login, and register test accounts.
    - Activate X Developer Basic Tier subscription ($100/month) and obtain OAuth 2.0 PKCE credentials.
    - Register Reddit application credentials for non-commercial script access.
- **Dependencies**: Corporate email verification and billing authorization.
- **Acceptance Criteria**: Verification via manual cURL calls that authorization codes can be exchanged for access tokens across all platforms.
- **Risks**: Delays or rejection in LinkedIn Community Management API vetting.
- **Estimated Effort**: 3 calendar weeks (gated by third-party review timelines).

Milestone 1: Core Foundation, Persistence & Security Shell

- **Deliverables**: Deploy PostgreSQL relational schemas, set up partitioned time-series snapshot tables, configure Redis task queues, build AES-256-GCM credential encryption services, and scaffold the FastAPI application structure.
- **Dependencies**: Completion of Milestone 0.
- **Acceptance Criteria**: Database migrations run cleanly; test suites confirm token encryption and decryption; Redis worker executes scheduled background tasks.
- **Risks**: Configuration errors in PostgreSQL partition schemas.
- **Estimated Effort**: 2 weeks.

Milestone 2: Read-Only MVP & Normalization Engine

- **Deliverables**: Implement OAuth 2.0 PKCE and authorization code flows for LinkedIn, X, Meta, and Reddit. Build public RSS feed ingestion for Substack and create a file upload drop-zone for creator CSV exports. Implement normalizers mapping external payloads to standard post and metric entities. Build the Next.js Content Library interface.
- **Dependencies**: Completion of Milestone 1.
- **Acceptance Criteria**: Ingestion of post history across all platforms; valid cross-platform post listings; appropriate em-dash rendering for unsupported metrics.
- **Risks**: Rate-limit exhaustion during initial historical backfill on X Basic tier.
- **Estimated Effort**: 3 weeks.

Milestone 3: Longitudinal Analytics & Snapshot Time-Series

- **Deliverables**: Implement the tiered exponential backoff polling worker in Redis/ARQ. Deploy webhook receiver endpoints for Meta platforms. Build the Overview Dashboard and Analytics views with comparative time-series charts. Implement aggregation queries with platform exclusion labeling.
- **Dependencies**: Completion of Milestone 2.
- **Acceptance Criteria**: Automated recurring workers log snapshots to post_metrics_snapshots; performance velocity charts render without double-counting.
- **Risks**: Inconsistent webhook delivery from Meta endpoints.
- **Estimated Effort**: 2.5 weeks.

Milestone 4: AI Intelligence & Insights Engine

- **Deliverables**: Build the deterministic analytical pre-processor to compute post rankings and format performance. Implement prompt injection defenses by isolating untrusted text within XML tags. Implement Pydantic-validated LLM extraction using external AI APIs. Deploy the AI Insights Studio interface.
- **Dependencies**: Completion of Milestone 3.
- **Acceptance Criteria**: LLM responses match target JSON schemas with zero validation failures; adversarial prompt injection attempts in comment strings fail to modify system instructions.
- **Risks**: Changes in upstream LLM API pricing or response formats.
- **Estimated Effort**: 2 weeks.

Milestone 5: Publishing Engine & Cross-Platform Calendar

- **Deliverables**: Build publishing adapters for Meta (asynchronous container flow), LinkedIn (/rest/posts), X (POST /2/tweets), and Reddit (/api/submit). Deploy the Publishing Calendar interface with cross-posting composers, custom platform overrides, and manual Substack dispatch checklists.
- **Dependencies**: Completion of Milestone 2.
- **Acceptance Criteria**: Programmatic publishing of text, single images, and PDF carousels to supported platforms directly from the calendar interface.
- **Risks**: Silent media upload failures on third-party endpoints.
- **Estimated Effort**: 2.5 weeks.

Milestone 6: Hardening, Observability & Production Readiness

- **Deliverables**: Set up Prometheus metric exporters and OpenTelemetry tracing for API rate limits and worker queues. Implement automated alerts for expiring tokens and failed sync jobs. Assemble end-to-end test suites using recorded API fixtures (VCR.py).
- **Dependencies**: Completion of Milestones 4 and 5.
- **Acceptance Criteria**: Test coverage exceeding 85% across platform adapters; zero data corruption during simulated Redis worker restarts.
- **Risks**: Platform API deprecations or schema changes.
- **Estimated Effort**: 2 weeks.

Deliverable 10: Initial Implementation Backlog

The following engineering backlog outlines the initial implementation tasks for Milestones 0 through 2, structured for direct conversion into development issues.

Task ID	Epic	Title & Description	Priority	Dependencies	Acceptance Criteria	Complexity	Reference
TSK-001	Feasibility	Submit LinkedIn Community Management API Vetting Request	P0	None	Developer application submitted in Microsoft Learn portal with verified company page and corporate email domain.	High	
TSK-002	Feasibility	Create Meta Developer App & Configure Unified Login	P0	None	App created with Instagram and Facebook products; test accounts added; initial test tokens generated.	Medium	
TSK-003	Feasibility	Subscribe to X API v2 Basic Tier & Provision Keys	P0	None	Basic tier ($100/mo) active; OAuth 2.0 Client ID and Secret obtained and stored in environment vault.	Low	
TSK-004	Core DB	Deploy PostgreSQL DDL & Partitioned	P0	None	Database migrations executed; partition	Medium	Deliverable 6

Task ID	Epic	Title & Description	Priority	Dependencies	Acceptance Criteria	Complexity	Reference
		Snapshot Tables			tables for 2026 deployed; foreign key cascades verified.		
TSK-005	Security	Implement AES-256-GCM Token Encryption Service	P0	TSK-004	Unit tests pass verifying encrypt/decrypt roundtrips and key derivation with authentication tags.	Medium	
TSK-006	Framework	Scaffold FastAPI Modular Backend & Adapter Base	P0	None	Abstract class SocialPlatformAdapter defined with standard async interface methods.	Low	ADR-001
TSK-007	Integration	Implement Meta OAuth & Unified Login Handler	P1	TSK-002, TSK-005	OAuth code flow exchanges long-lived page and Instagram user tokens; records stored encrypted.	High	
TSK-008	Integration	Implement X API v2 OAuth 2.0 PKCE Flow	P1	TSK-003, TSK-005	PKCE authorization code exchange operational; offline refresh	Medium	

Task ID	Epic	Title & Description	Priority	Dependencies	Acceptance Criteria	Complexity	Reference
					tokens automatically handled.		
TSK-009	Integration	Implement Reddit OAuth 2.0 Flow	P1	TSK-005	Script and web app authorization functional; permanent refresh tokens acquired.	Low	
TSK-010	Integration	Implement Substack Public RSS Ingestion Worker	P1	TSK-004	Parser reads substack.com/feed, extracts posts, canonical links, HTML bodies; upserts to posts.	Low	
TSK-011	Integration	Implement Substack CSV Export Ingestion Parser	P1	TSK-004	Parser processes posts.csv and subscribers.csv; maps private open rates and subscriber counts.	Medium	
TSK-012	Ingestion	Build LinkedIn Post Normalizer & Metrics Collector	P1	TSK-001, TSK-005	Fetches posts via /rest/posts and metrics via memberCreatorPostAnalytics; handles DAILY	High	

Task ID	Epic	Title & Description	Priority	Dependencies	Acceptance Criteria	Complexity	Reference
					aggregation limits.		
TSK-013	Ingestion	Build X v2 Timeline Normalizer & Metrics Poller	P1	TSK-008	Polls /2/users/:id/tweets; extracts organic impressions, bookmarks, likes, quotes; logs snapshots.	Medium	
TSK-014	Ingestion	Build Instagram Media & Insights Ingestion Worker	P1	TSK-007	Queries /insights for impressions, reach, plays, saves; normalizes media objects; logs snapshots.	Medium	
TSK-015	Ingestion	Build Facebook Page Ingestion Worker	P1	TSK-007	Fetches /published_posts and post insights; stores permanent page access tokens.	Low	
TSK-016	Frontend	Develop Unified Content Library & Filtering UI	P1	TSK-004, TSK-006	Next.js table component displays normalized posts; multi-platform filter and sorting fully functional.	Medium	Deliverable 8

Deliverable 11: Open Questions and Unresolved Risks

1. **LinkedIn Community Management API Approval Timeline**:
    - *Uncertainty*: LinkedIn does not publish standard review timelines for Community Management API access; approval may take days, weeks, or face rejection based on business verification criteria.
    - *Impact*: If rejected, mTools can publish member posts via open w_member_social, but personal post historical reading and analytics retrieval will be blocked.
    - *Resolution Strategy*: Submit the application during Milestone 0 using an established corporate email domain and verified company page. Keep the LinkedIn analytics adapter behind a feature flag during initial rollout.
    - *Implementation Blocker*: Non-blocking for the core application build; blocking for LinkedIn personal analytics.
2. **X API Pricing and Policy Stability**:
    - *Uncertainty*: X has restructured API pricing and rate limits multiple times over recent years with little notice.
    - *Impact*: Any deprecation of the $100/month Basic tier or further reduction of its 10,000 monthly read limit could make reading post analytics financially impractical for a personal dashboard.
    - *Resolution Strategy*: Track official X developer announcements and implement aggressive Redis caching to minimize read requests.
    - *Implementation Blocker*: Non-blocking, but represents an ongoing operational cost risk.
3. **Substack CSV Export Schema Consistency**:
    - *Uncertainty*: Because Substack does not offer an official developer API, export formats (posts.csv, subscribers.csv) are internal product features subject to unannounced header or formatting changes.
    - *Impact*: Unannounced schema changes could cause CSV ingestion jobs to fail or misalign data fields.
    - *Resolution Strategy*: Implement schema-tolerant CSV parsing using Pydantic models with column alias mappings and automated validation alerts.
    - *Implementation Blocker*: Non-blocking.
4. **Reddit REST API Analytics Availability**:
    - *Uncertainty*: Reddit continues to direct post analytics (Post Insights) through internal GraphQL endpoints (gql.reddit.com) rather than its public REST API.
    - *Impact*: View counts and organic impressions will remain unavailable on standard Reddit REST endpoints.
    - *Resolution Strategy*: Accept that Reddit cannot contribute to impression metrics; clearly indicate in the UI that Reddit tracks only karma scores and comment volume.
    - *Implementation Blocker*: Non-blocking.
5. **Meta Token Revocation Cascades**:
    - *Uncertainty*: While Facebook Page tokens obtained from long-lived user tokens do not expire by default, user-initiated security actions (such as account password changes) invalidate all downstream tokens immediately.
    - *Impact*: Background synchronization jobs will fail with OAuth authentication errors until the user logs in again.

- *Resolution Strategy*: Implement proactive token health monitoring that detects invalid token states and prompts the user to re-authenticate.
- *Implementation Blocker*: Non-blocking.

Deliverable 12: Operating Cost and Financial Feasibility Analysis

Operating costs for mTools depend primarily on third-party developer platform fees, cloud infrastructure, and AI inference workloads. The estimates below compare a single-user personal deployment against a multi-account growth scenario.

Cost Category	Personal Deployment (Single User, 6 Accounts)	Multi-Account / Growth Deployment (5 Users, 25 Accounts)	Cost Drivers & Billing Notes
X API Access	$100.00 / month	$100.00 – $5,000.00 / month	Basic tier ($100/mo) covers up to 10, 5,000/mo).
LinkedIn API Access	$0.00	$0.00	Free access under the Community Management API, subject to application approval.
Meta Graph API Access	$0.00	$0.00	Free access for Instagram Professional and Facebook Pages.
Reddit Data API Access	$0.00	$0.00	Free for non-commercial use within 100 queries/minute.
Substack Ingestion	$0.00	$0.00	Public RSS polling and local CSV parsing incur no API fees.
Cloud Hosting & Containers	$15.00 – $25.00 / month	$60.00 – $120.00 / month	Managed container hosting (Render, Railway, or Fly.io) for FastAPI backend and Next.js frontend.
Managed Database & Redis	$15.00 – $25.00 / month	$50.00 – $100.00 / month	Managed PostgreSQL (Supabase, Neon, or RDS) and managed Redis (Upstash or Redis Cloud).
Object Storage	$1.00 – $3.00 / month	$5.00 – $15.00 / month	S3-compatible storage for media attachments and CSV archives.
LLM Inference (AI Insights)	$3.00 – $10.00 / month	$20.00 – $50.00 / month	OpenAI/Anthropic API tokens for weekly

Cost Category	Personal Deployment (Single User, 6 Accounts)	Multi-Account / Growth Deployment (5 Users, 25 Accounts)	Cost Drivers & Billing Notes
			analysis and comment summaries.
Total Estimated Cost	**~$134.00 – $163.00 / month**	**~$235.00 – $5,285.00 / month**	Dominated by X API fees in personal setups; growth scaling depends on X tier limits.

Deliverable 13: Failure Modes, Security, and Observability Architecture

Failure Mode Mitigations

- **OAuth Token Expiration & Revocation**: Tokens are verified daily using a background health check. If an access token expires within 7 days and a refresh token is present, the worker refreshes the credential automatically. If an integration returns an authentication failure, the integration status is set to NEEDS_REAUTH, and an in-app alert is generated.
- **Rate-Limit Depletion**: Each outbound request updates a rate-limit tracking key in Redis based on response headers (such as x-rate-limit-remaining on X or x-app-usage on Meta). When remaining quota drops below 10%, scheduled polling tasks enter a cooldown period until the platform reset timestamp.
- **Transient Outages**: External requests use an exponential backoff policy (2^n \times \text{base_delay}) with random jitter for 5xx server responses, retrying up to four times before routing the job to a dead-letter queue.
- **Content Deletion & Edits**: If a post returns an HTTP 404 during synchronization, is_deleted_on_platform is set to TRUE, preserving historical metric snapshots in the database for longitudinal reporting. When post text is modified upstream, the posts record updates content_text and logs the edit event in audit_logs.

Security and Data Protection

- **Credential Isolation**: Access and refresh tokens are encrypted at rest using AES-256-GCM. Encryption keys are injected via environment variables and tracked using an encryption_key_id to facilitate zero-downtime key rotation. Decrypted tokens exist only in worker memory during request execution and are excluded from logs and API responses.
- **Prompt Injection Defense**: Untrusted content (such as post commentary and incoming replies) is stripped of control characters, truncated to 1,000 characters, and enclosed in explicit XML tags (<untrusted_social_text>) within LLM prompts. System prompts instruct models to treat text within these tags as passive data, and model outputs are validated against Pydantic schemas.
- **Webhook Signature Verification**: Webhook endpoints validate HMAC signatures (such as X-Hub-Signature-256 on Meta) before parsing payloads. Requests with invalid or missing signatures are rejected with HTTP 401.

Observability Standards

- **Structured Logging**: All application services output JSON logs containing timestamp, log_level, integration_id, job_id, and trace_id.
- **Telemetry & Metrics**: OpenTelemetry instrumentation tracks external API request durations, rate-limit consumption rates, worker queue depths, and snapshot ingestion volumes.
- **Reconciliation Jobs**: A weekly reconciliation task checks for discrepancies between current platform metrics and database records, correcting drift caused by dropped webhook events.

Deliverable 14: Testing Strategy

The test plan evaluates system reliability across normalization logic, external API adapters, authentication flows, and data security without requiring live production social accounts for routine testing.

```
[Test Suite Hierarchy]
├── Unit Tests: Payload normalization, metric formulas, prompt sanitization
├── Contract Tests: Recorded VCR.py fixtures validating platform API responses
├── Integration Tests: Mocked OAuth handshakes, token refreshes, rate-limit backoff
└── Security Tests: Encryption roundtrips, signature verification, injection defense
```

1. Normalization Unit Tests

- Validate that raw JSON payloads from all six platforms map accurately to the normalized posts and post_metrics_snapshots schemas.
- Confirm that derived metrics (such as engagement rates) calculate accurately across various engagement scenarios.
- Confirm that missing platform metrics (such as impressions on Reddit or reach on X) produce null values rather than defaulting to zero.

2. Adapter Contract & Mocking Tests

- Use recorded VCR.py fixtures to simulate external API responses across standard responses, error states, and pagination flows.
- Simulate HTTP 429 rate-limit responses and verify that the adapter parses reset headers and sets worker cooldown timers correctly.
- Simulate HTTP 401 unauthorized errors and verify that the adapter initiates token refresh flows.

3. Ingestion & Concurrency Tests

- Verify that recurring synchronization tasks use idempotent upserts, preventing duplicate rows in posts or snapshot tables during overlapping job runs.
- Test partition management by writing snapshots across past, current, and future quarterly partition ranges.
- Test Substack CSV ingestion by parsing sample posts.csv and su[span_125](start_span)[span_125](end_span)[span_134](start_span)[span_134](end_span)[span_143](start_span)[span_143](end_span)bscribers.csv files with missing columns, unexpected headers, and varied date formats .

4. Security & Prompt Injection Tests

- Run unit tests on the AES-256-GCM encryption service to confirm ciphertext security, authentication tag validation, and multi-key rotation.
- Validate that webhook endpoints reject payloads with invalid HMAC signatures.
- Submit adversarial prompt injection payloads (such as instructions to ignore system prompts) through the comment ingestion pipeline and verify that the LLM output remains confined to the expected schema.

Synthesis and Concluding Recommendations

1. Smallest Useful Version of mTools That Can Be Built Reliably

The smallest viable release is a **Read-Only Unified Dashboard and Content Library** integrating **Instagram Professional**, **Facebook Pages**, and **X (Basic Tier)** via official APIs, alongside automated **Substack Public RSS indexing**. This MVP delivers unified content history, genuine impression-based engagement tracking across the primary social web, and publishing consistency monitoring without being blocked by enterprise application reviews.

2. Platform Integration Sequence

1. **Instagram Professional & Facebook Pages**: Free developer access, unified Meta Graph architecture, comprehensive metrics, and immediate availability in Development mode.
2. **Substack (RSS Track)**: No authorization hurdles, open XML ingestion, and immediate content indexing.
3. **X (Twitter)**: Standard OAuth 2.0 PKCE implementation, fully operational once the $100/month Basic tier is subscribed.
4. **Reddit**: Straightforward OAuth 2.0 configuration; immediate access to submission history and community discussions.
5. **Substack (CSV Upload Track)**: Manual file ingestion interface to add private subscriber and email open analytics.
6. **LinkedIn**: Dependent on external Community Management API application review and corporate vetting.

3. Genuinely Unifiable Metrics

Only three engagement metrics can be aggregated across all platforms without conceptual distortion:

- **Gross Reactions / Likes**: LinkedIn REACTION, X like_count, Instagram likes, Facebook reactions, Reddit score (net), Substack likes.
- **Comment / Discussion Volume**: Total comment and reply counts across all platforms (excluding Substack when API access is absent).
- **Publication Velocity**: Timestamped count of published content across all connected channels.

Note on Impressions: Impressions can be aggregated across LinkedIn, X, Instagram, and Facebook, but **must explicitly exclude Reddit and RSS-based Substack**.

4. Metrics That Must Remain Platform-Specific

- **Reddit Net Score & Upvote Ratio**: Reflects community voting behavior and downvote subtractions unique to Reddit.
- **LinkedIn Member Sends & Profile Views from Content**: Private professional distribution metrics with no equivalent on other platforms.
- **X Bookmark Count**: Private content curation metrics specific to X.
- **Instagram Saves & Profile Activity**: Visual bookmarking and mobile bio link navigation metrics.
- **Substack Email Open Rates & Gross Annualized Revenue**: Newsletter delivery metrics and direct creator subscription finances.

5. Integrations Requiring Developer Approval or Paid Access

- **LinkedIn Community Management API**: Requires business email verification, verified company page administration, and formal application review.
- **X API v2**: Requires a recurring commercial subscription ($100/month Basic tier) to access read endpoints and post analytics.
- **Meta Advanced Access**: Required if the application is distributed beyond 50 internal test accounts; requires Meta App Review and Business Verification.

6. Features Deferred as Inaccessible or Cost-Prohibitive

- **Reddit Post Impression Tracking**: Inaccessible via public REST APIs; restricted to internal first-party GraphQL queries.
- **Substack Native API Publishing & Real-Time Sync**: Deferred entirely due to the absence of a public developer API.
- **Real-Time Webhooks for X and LinkedIn Personal Profiles**: Deferred because X Account Activity webhooks require the $5,000/month Pro tier and LinkedIn personal webhooks are unavailable; periodic polling is used instead.

7. Recommended Architecture

A **Modular Monolith** built using **Python 3.12 (FastAPI)**, managed **PostgreSQL 16** with

monthly declarative table partitioning for metric snapshots, **Redis 7** with **ARQ** for asynchronous task execution, and **Next.js 14** on the frontend. This provides high maintainability for personal use while allowing connectors to scale independently.

8. Major Recurring Operating Costs

- **X API v2 Basic Tier**: $100.00 / month (fixed platform fee).
- **Managed Database & Hosting (e.g., Supabase / Render)**: ~$25.00 – $50.00 / month.
- **LLM Inference (OpenAI / Anthropic structured insights)**: ~$5.00 – $15.00 / month.
- **Total Estimated Monthly Budget**: **~$130.00 to $165.00 per month**.

9. First Ten Engineering Tasks

1. Register application in Meta Developer Portal and configure Unified Login with Instagram/Facebook test accounts. 2. Submit vetting application for LinkedIn Community Management API via LinkedIn Company Page. 3. Provision X Developer Basic Tier subscription ($100/mo) and generate v2 OAuth 2.0 PKCE credentials. 4. Deploy local PostgreSQL 16 instance and execute schema migrations for core tables and snapshot partitions (Deliverable 6). 5. Implement AES-256-GCM encryption service for OAuth credential storage. 6. Scaffold FastAPI application structure and define SocialPlatformAdapter abstract interface. 7. Implement Substack RSS 2.0 XML polling worker to establish initial content ingestion. 8. Implement Meta OAuth 2.0 authentication handler and persist 60-day Instagram and non-expiring Page tokens. 9. Implement X API v2 OAuth 2.0 PKCE flow and build timeline polling worker for /2/users/:id[span_858](start_span)[span_858](end_span)/tweets. 10. Construct basic Next.js Content Library interface to render ingested post objects and verify cross-platform normalization.

10. Items to Verify Before Implementation Begins

1. **Confirm LinkedIn Corporate Eligibility**: Ensure an active LinkedIn Company Page exists with administrative rights and a matching business email domain to satisfy Community Management API vetting requirements.
2. **Commitment to X API Monthly Spend**: Validate budget authorization for the recurring $100/month X developer fee.
3. **Instagram Account Conversion**: Confirm the personal Instagram account has been converted to a Professional Account (Creator or Business) within the Instagram mobile application.
4. **Substack Workflow Acceptance**: Confirm acceptance of the hybrid integration model (automated RSS for public posts; periodic manual CSV uploads for email and subscriber metrics).

Works cited

1. Instagram Media Insights | Meta for Developers - Facebook, https://developers.facebook.com/documentation/instagram-platform/reference/instagram-media/insights 2. Instagram Platform insights | Meta for Developers - Facebook, https://developers.facebook.com/documentation/instagram-platform/insights 3. Instagram API in 2026: The Complete Developer Guide - Upload-Post,

https://www.upload-post.com/instagram-api/ 4. Facebook & Instagram Long-Lived Page Access Tokens (2026), https://www.outstand.so/blog/facebook-instagram-long-lived-page-access-token 5. Non Ad Post Views/Impressions API Endpoint - Reddit, https://www.reddit.com/r/redditdev/comments/1i22pdj/non_ad_post_viewsimpressions_api_endpoint/ 6. API view count request : r/redditdev, https://www.reddit.com/r/redditdev/comments/6v2fmp/api_view_count_request/ 7. Twitter bot: build an AI-powered bot for X - CodeWords, https://www.codewords.ai/blog/twitter-creator-bot 8. How to Connect Twitter/X to OpenClaw: Automated Social Media, https://sfailabs.com/guides/connect-twitter-to-openclaw 9. Tweet Scraper: Extract X/Twitter Data in Python - ScrapeGraphAI, https://scrapegraphai.com/blog/tweet-scraper 10. How to export Substack posts to Jekyll - Uncategorized, https://talk.jekyllrb.com/t/how-to-export-substack-posts-to-jekyll/7683 11. Substack Estate Planning: What to Set Up Now - SimplyTrust, https://simplytrust.com/digital-assets/substack/planning-your-estate/ 12. Import Substack Newsletter/Blog Posts into Pelican Posts Folder, https://neolateral.in/import-substack-newsletterblog-posts-into-pelican-posts-folder 13. Community Management - Overview - LinkedIn - Microsoft Learn, https://learn.microsoft.com/en-us/linkedin/marketing/community-management/community-management-overview?view=li-lms-2026-09 14. Post to a LinkedIn Company Page via API (No Partner Program), https://www.upload-post.com/how-to/post-to-linkedin-company-page-api/ 15. What Is the LinkedIn API? Complete Guide for 2026 - CompLeadly, https://compleadly.com/blog/what-is-the-linkedin-api-a-complete-guide-for-2026 16. How to Use the X (Twitter) API to Grow and Automate Your Presence, https://tweetfull.com/blog/api-twitter-how-to-use-the-x-twitter-api-to-grow-and-automate-your-presence/ 17. X - Grokipedia, https://grokipedia.com/page/Twitter 18. LinkedIn Analytics API: Metrics, OAuth & Code Examples [2026], https://zernio.com/blog/linkedin-analytics-api 19. LinkedIn API Access for Posting: Profile vs Page - LotsSocial Blog, https://blog.lots.social/linkedin-api-posting-profile-vs-page 20. LinkedIn posting API for AI agents - PostLake, https://postlake.dev/post-to/linkedin 21. Linkedin Api (Grade B) - Claude Skill - Skills Directory, https://www.skillsdirectory.com/skills/ericrisco-linkedin-api 22. Member Post Statistics - LinkedIn - Microsoft Learn, https://learn.microsoft.com/en-us/linkedin/marketing/community-management/members/post-statistics?view=li-lms-2026-09 23. Recent Marketing API Changes - LinkedIn - Microsoft Learn, https://learn.microsoft.com/en-us/linkedin/marketing/integrations/recent-changes?view=li-lms-2026-10 24. LinkedIn API Integration in Python Django (OAuth 2.0 + OIDC), https://micropyramid.com/blog/integration-of-linkedin-api-in-python-django/ 25. instagram-oauth.md - oomol-lab/open-connector - GitHub, https://github.com/oomol-lab/open-connector/blob/main/docs/instagram-oauth.md 26. Migrate your App | Meta for Developers - Facebook, https://developers.facebook.com/documentation/instagram-platform/instagram-api-with-instagram-login/migration-guide 27. Migrate your App - Meta for Developers, https://developers-fallback.facebook.com/documentation/instagram-platform/instagram-api-with-instagram-login/migration-guide.md 28. v2.3 - Graph API - Meta for Developers, https://developers.facebook.com/docs/graph-api/changelog/version2.3/ 29. Object Sharedposts - Graph API - Meta for Developers, https://developers.facebook.com/docs/graph-api/reference/object/sharedposts/ 30. Graph API Reference v26.0: Post - Meta for Developers, https://developers.facebook.com/docs/graph-api/reference/post/ 31. Help Center - Using

Insights in Buffer, https://support.buffer.com/articles/using-insights-in-buffer-x4gLauQU5a 32. Substack Scraper - Export Newsletter Posts MCP server · Apify, https://apify.com/nexo_data/substack-public-archive-and-changes/api/mcp 33. Creators – Substack, Inc, https://support.substack.com/hc/en-us/categories/360002403472-Creators 34. How can I publish on Substack?, https://support.substack.com/hc/en-us/articles/29152946791188-How-can-I-publish-on-Substack 35. API for Post Insights - r/redditdev on Reddit, https://www.reddit.com/r/redditdev/comments/tn1yvp/api_for_post_insights/ 36. Office Hours: Obstacles and support 🏔️ - On Substack, https://on.substack.com/p/office-hours-services/comments 37. How do I use the subscriber dashboard on Substack?, https://support.substack.com/hc/en-us/articles/360058529871-How-do-I-use-the-subscriber-dashboard-on-Substack 38. Technical Protocol: How to Work with Substack Export - Lintara, https://lintara.online/technical-protocol-work-substack-export/ 39. substack-subscriber-analytics/data_loader.py at main - GitHub, https://github.com/denominations-io/substack-subscriber-analytics/blob/main/data_loader.py 40. Move from Substack to sparx · sparx, https://sparx.works/migrate/substack